ASGI Specification v3.0

Key Points Reference

Date: 2026-08-02

Source: https://asgi.readthedocs.io/en/latest/specs/main.html

Sub-specs: /specs/www.html | /specs/lifespan.html

---

1. Overview

* ASGI (Asynchronous Server Gateway Interface) is the successor to WSGI, supporting HTTP/1.1, HTTP/2, and WebSocket in one unified interface.
* Spec version 3.0 (2019-03-20). Sub-specs: HTTP & WebSocket v2.5 (2024-06-05), Lifespan v2.0 (2019-03-20).
* WSGI was tied to HTTP request/response cycles; ASGI allows data to be sent/received at any time, supporting long-lived connections.
* ASGI consists of two components:
  - Protocol server -- terminates sockets, translates them into connections and per-connection event messages.
  - Application -- lives inside the server, called once per connection, handles events and emits responses.
* Applications must be async/await coroutines (asyncio-compatible) on the main thread; sync work may be offloaded to threads or processes.
* Unlike WSGI's single request-in/response-out model, an ASGI connection has two parts: a connection scope (metadata) and events (the data stream).
* Each call of the application callable maps to a single incoming socket/connection and lasts its full lifetime.

2. Application -- The Async Callable (v3.0 single-callable style)

The application is a single async callable:

```
async def application(scope, receive, send):
    ...
```

* scope -- dict with at least 'type' key identifying the protocol ('http', 'websocket', 'lifespan'). Also contains 'asgi' sub-dict.
* receive -- awaitable callable; awaiting yields the next incoming event dict when available.
* send -- awaitable callable; accepts a single event dict, returns when send completes or connection closes.
* Called exactly once per connection. Connection lifetime is protocol-specific (one HTTP request, one WebSocket, one lifespan per event loop).
* scope['asgi']['version'] -- ASGI version the server implements. Missing => default '2.0'.
* scope['asgi']['spec_version'] -- optional; per-protocol spec versioning independent of top-level ASGI version.
* scope['type'] is ALWAYS present; use it to determine which protocol handler to invoke.

Legacy v2.0 Applications (two-callable style -- retired)

* Old style: synchronous application(scope) returns awaitable application_instance(receive, send).

* Retired in v3.0. Servers should support both via asgiref.compatibility module and gradually drop legacy support.

3. Connection Scope

* A dict passed as the first argument to the application, describing the full connection context.
* Survives for the lifetime of the connection (single HTTP request, full WebSocket session, or full lifespan event loop).
* scope['type'] is always present; drives protocol dispatch.
* HTTP scope: short-lived (one request). Most request metadata is in scope; body arrives via events (not in scope).
* WebSocket scope: lives as long as the socket. Path and headers are in scope; messages arrive as events.
* Lifespan scope: lives for the full event loop duration.
* Middleware must copy scope before mutating and passing to inner app -- mutations would otherwise leak upstream.
* scope['extensions'] -- optional dict mapping extension names to dicts, signalling optional server capabilities.

4. Events (receive / send)

* Every event is a dict with a mandatory 'type' Unicode string key (e.g. 'http.request', 'websocket.send').
* Naming convention: 'protocol.message_type' where protocol matches scope['type'] (e.g. 'http.request', 'websocket.receive').
* Custom event types are allowed for application-level messaging (e.g. 'mychat.message').
* Events must be serializable. Permitted value types only:
  - byte strings, Unicode strings
  - integers (signed 64-bit), floats (IEEE 754 double -- no NaN or Infinity)
  - lists (tuples must be encoded as lists), dicts (keys must be Unicode strings)
  - booleans, None
* Extra/unknown keys in event dicts MUST NOT raise exceptions -- enables forward-compatible spec evolution.
* Invalid events: server raises exception from send(); application should raise on bad receive() data.
* Apps SHOULD reject unknown scope['type'] values with an Exception to avoid silent misconfiguration.

5. HTTP Protocol Sub-Spec (scope type: 'http')

Connection Scope Keys

* type: 'http'
* asgi['version'] (str), asgi['spec_version'] (str, optional)
* http_version: '1.0', '1.1', or '2' (Unicode str)
* method: HTTP method uppercased (e.g. 'GET', 'POST')
* scheme: 'http' or 'https' (default 'http')
* path: percent-decoded request path, no query string
* raw_path: original bytes path unmodified (optional; default None)
* query_string: bytes of URL query string, percent-encoded
* root_path: mount point equivalent to WSGI SCRIPT_NAME (default '')
* headers: list of [name_bytes, value_bytes] pairs; order and duplicates preserved; names should be

lowercased

* client: [host_str, port_int] (optional; default None)
* server: [host_str, port_int] or [unix_path, None] (optional; default None)
* state: shallow copy of lifespan state namespace (optional; server feature)

HTTP Events

* http.request (receive) -- body: bytes (default b''), more_body: bool. Stream large bodies chunk-by-chunk with more_body=True.
* http.disconnect (receive) -- fires after response completes or client disconnects. Useful for long-poll cleanup.
* http.response.start (send) -- status: int, headers: [[name_bytes, value_bytes], ...], trailers: bool (default False).
* http.response.body (send) -- body: bytes, more_body: bool. Final chunk has more_body=False, closing the response.

HTTP Implementation Notes

* Chunked Transfer-Encoding is handled entirely by the server; apps receive and send plain body bytes.
* Apps MUST NOT set Transfer-Encoding response header. Content-Encoding (gzip etc.) is still the app's domain.
* Set-Cookie headers must NOT be comma-concatenated; pass them as separate list entries.
* HTTP/2: server multiplexes responses back to correct stream. Cookie header may appear multiple times per RFC 9113.
* send() on closed connection raises a server-specific OSError subclass (introduced in spec 2.4; not guaranteed on older servers).
* Response start may be buffered until the first response body event -- allows replacing it with an error if needed.

6. WebSocket Protocol Sub-Spec (scope type: 'websocket')

Connection Scope Keys

* type: 'websocket'
* scheme: 'ws' or 'wss' (default 'ws')
* path, raw_path, query_string, root_path -- same semantics as HTTP
* headers: same as HTTP scope (list of [name_bytes, value_bytes] pairs)
* subprotocols: list of Unicode strings advertised by client (default [])
* state: lifespan state shallow copy (optional)
* Scope lifetime == socket lifetime; if app dies, server closes socket.

WebSocket Event Flow (in order)

* 1. websocket.connect (receive) -- client initiating handshake; received BEFORE handshake completes. App must respond.
* 2. websocket.accept (send) -- accepts handshake. Keys: subprotocol (str, optional), headers (list, optional, spec 2.1+).
* 3. websocket.receive (receive) -- data from client. Keys: bytes (binary) or text (Unicode); exactly one must be non-None.
* 4. websocket.send (send) -- data to client. Keys: bytes or text; exactly one must be non-None.
* 5. websocket.disconnect (receive) -- connection lost. Keys: code (int, default 1005), reason (str, spec

2.5+).

* websocket.close (send) -- close from app side. Keys: code (default 1000), reason. Sent before accept => server returns HTTP 403.

WebSocket Implementation Notes

* PING/PONG frames handled automatically by the server; apps need not implement them.
* Message fragmentation handled by the server; apps always receive complete messages.
* send() on closed connection raises server-specific OSError subclass (spec 2.4+).
* Calling websocket.close BEFORE websocket.accept causes server to return HTTP 403 Forbidden to client.

7. Lifespan Protocol Sub-Spec (scope type: 'lifespan')

* Allows application startup/shutdown logic to run in the same event loop that handles requests.
* Critical use case: create a DB connection pool at startup, close it at shutdown, share it with request handlers via scope state.
* Executed once per event loop -- one per process (multi-process), one per thread (multi-threaded).
* scope keys: type='lifespan', asgi['version'], asgi['spec_version'] (optional, default '1.0'), state (dict, optional).

Lifespan Events

* lifespan.startup (receive) -- server is ready to start but has NOT yet accepted connections.
* lifespan.startup.complete (send) -- app ready; server MUST wait for this before accepting connections.
* lifespan.startup.failed (send) -- startup failed; server logs 'message' key and exits. Prevents server from starting.
* lifespan.shutdown (receive) -- server has stopped accepting connections and closed active ones.
* lifespan.shutdown.complete (send) -- cleanup done; server MUST wait for this before terminating.
* lifespan.shutdown.failed (send) -- cleanup failed; server logs 'message' key and terminates.

Lifespan State

* scope['state'] is a plain dict controlled entirely by the app. Server passes a shallow copy to every HTTP/WebSocket request scope.
* Eliminates need for global mutable state or context variables for objects like DB connection pools.
* Apps should namespace state keys (e.g. 'myapp.db_pool') to avoid middleware collisions.

Compatibility

* If the app raises an exception on a 'lifespan' scope call, server continues WITHOUT sending lifespan events -- backward compatible with apps that don't implement lifespan.
* To intentionally fail startup (block server start), send lifespan.startup.failed -- do NOT just raise an exception.

8. Middleware

* Middleware sits between the outer server and an inner application, acting as both.
* Receives (scope, receive, send), may modify them, then calls the inner app.
* MUST copy scope before mutating -- changes to the original leak upstream to callers holding references.
* After handing scope to the inner app, do not retain a reference and try to mutate it -- you have one chance at call time.

* Middleware can wrap receive/send callables to intercept and transform events in both directions.

9. Extensions Mechanism

* scope['extensions'] -- optional dict of server-supported extension names -> dict values.
* An empty dict value signals the server can handle a custom send event of a given type.
* Non-empty dict values can carry extra scope metadata specific to the extension.
* Example: extensions={'fullflush': {}} tells the app it may send 'http.fullflush' events.
* Intended for server-specific additions and trialling changes before formal inclusion in the spec.

10. Error Handling Rules

* Server receives invalid event from app via send(): raise exception back into the app.
* App receives invalid event from receive(): app should raise an exception.
* Extra/unknown keys in event dicts MUST NOT raise -- forward compatibility rule.
* Server must terminate the app instance and connection if an uncaught exception bubbles up.
* Post-close send() calls are no-ops unless the protocol spec specifies otherwise.
* Servers must catch errors raised by their own send() exception and not propagate them as server errors.

11. Type Conventions

* 'byte string' = Python bytes; 'Unicode string' = Python str. The spec never uses bare 'string'.
* All dict keys in scopes and events are Unicode strings.
* Headers always transported as list of [name_bytes, value_bytes] pairs to preserve order and duplicates.
* Tuples must be encoded as lists in event dicts for serialization across processes/network.

12. Quick Reference -- Event Type Summary

Protocol	Direction	Event type	Purpose
HTTP	receive	http.request	Incoming request body (streamable)
HTTP	receive	http.disconnect	Client disconnected / response done
HTTP	send	http.response.start	Begin response (status + headers)
HTTP	send	http.response.body	Response body chunk
WebSocket	receive	websocket.connect	Client initiating handshake
WebSocket	send	websocket.accept	Accept handshake
WebSocket	receive	websocket.receive	Incoming data message
WebSocket	send	websocket.send	Outgoing data message
WebSocket	receive	websocket.disconnect	Connection closed
WebSocket	send	websocket.close	App closes connection
Lifespan	receive	lifespan.startup	Server about to start
Lifespan	send	lifespan.startup.complete	App startup complete
Lifespan	send	lifespan.startup.failed	App startup failed
Lifespan	receive	lifespan.shutdown	Server shutting down
Lifespan	send	lifespan.shutdown.complete	App cleanup complete
Lifespan	send	lifespan.shutdown.failed	App cleanup failed

ASGI Specification v3.0 -- Key Points

Sources: https://asgi.readthedocs.io/en/latest/specs/main.html | https://asgi.readthedocs.io/en/latest/specs/www.html | https://asgi.readthedocs.io/en/latest/specs/lifespan.html